

RELATÓRIO TÉCNICO-CIENTÍFICO: GRIFON v3.6.1 & A TEORIA DA COLORAÇÃO DE GRAFOS

Jailis Dourado

2026

Disciplina: Algoritmos em Grafos (7º período)

Instituto Federal Goiano

Orientador: Prof. André da Cunha Ribeiro

Resumo

Este relatório documenta o desenvolvimento do **Grifon - Grafo Interativo v3.6.1**, uma ferramenta *web* para visualização e **coloração total de grafos**. O sistema implementa algoritmos clássicos de coloração (*Welsh-Powell*, *round-robin*, *guloso*) com animações passo a passo, oferecendo uma plataforma didática para o ensino de teoria dos grafos. São discutidos os fundamentos teóricos, a arquitetura do sistema, as funcionalidades da interface, os testes realizados com diferentes tipos de grafos e as limitações técnicas da implementação.

Sumário

1	Introdução e Contexto Acadêmico	4
2	Fundamentação Teórica: O Problema da Coloração	4
2.1	Conceitos Básicos	4
2.2	Teoremas Fundamentais	5
3	Algoritmos de Coloração de Vértices	5
3.1	Algoritmo de <i>Welsh-Powell</i> Melhorado	5
3.2	Por que <i>Welsh-Powell</i> e não <i>DSATUR</i> ?	6
4	Algoritmos de Coloração de Arestas	6
4.1	Algoritmo <i>Round-Robin</i> (para grafos completos com n ímpar)	7
4.2	Esquema de Diagonais (para $K_{3,3}$)	7
4.3	Algoritmo Guloso Otimizado (caso geral)	7
5	Arquitetura e Tecnologias do Grifon	8
5.1	Principais Tecnologias	8
5.2	Estrutura do Código	8
5.3	Histórico (<i>Undo/Redo</i>)	9
6	Funcionalidades da Interface e Usabilidade	9
6.1	Visão Geral da Interface	9
6.2	Estatísticas em Tempo Real	10
6.3	Modos de Operação	10
7	Animações e Recursos Didáticos	10
7.1	Animação Passo a Passo da Coloração Total	10
7.2	Centralização Automática	10
7.3	<i>Logs</i> Didáticos	11
7.4	Modelos Pré-definidos	11
8	Gestão de Dados e Formato de Arquivo	11
8.1	Exportação para <i>.txt</i>	11
8.2	Importação de <i>.txt</i>	12
8.3	Pasta <i>/data</i>	12
9	Performance e Limitações Técnicas	12
9.1	Complexidade dos Algoritmos	13
9.2	Consumo de Memória	13
9.3	Gargalo do Algoritmo Guloso	13
10	Aplicações Práticas da Coloração de Grafos	13
10.1	Alocação de Registros em Compiladores	14
10.2	Escalonamento de Horários (Problema do Horário Escolar)	14
10.3	Malha de Transporte Público e Redes de Internet	14
10.4	Atribuição de Frequências em Redes de Celular	14
10.5	Otimização de Tráfego Aéreo	14

1 Introdução e Contexto Acadêmico

O presente relatório documenta o desenvolvimento do **Grifon - Grafo Interativo** em sua versão *v3.6.1*, uma ferramenta *web* para visualização e **coloração total de grafos**. O projeto foi concebido como parte do *Seminário II* da disciplina de Algoritmos em Grafos, do curso de Ciência da Computação do Instituto Federal Goiano, sob orientação do professor André da Cunha Ribeiro.

Desenvolvido integralmente por Jailis Dourado, o sistema tem como propósito unir o rigor matemático da Teoria dos Grafos com a interatividade das tecnologias *web* modernas. Diferentemente de outras ferramentas acadêmicas que se limitam à apresentação estática de resultados, o Grifon oferece **animações passo a passo**, permitindo que estudantes e pesquisadores visualizem a execução de algoritmos complexos de coloração de maneira dinâmica, controlada e didática.

O tema central do trabalho é a **coloração total de grafos** – um problema que consiste em atribuir cores a vértices e arestas de modo que:

- **Vértices adjacentes** não compartilhem a mesma cor;
- **Arestas adjacentes** (que compartilham um vértice) não compartilhem a mesma cor;
- Cada vértice tenha cor diferente das arestas que lhe são incidentes.

A escolha desse tema justifica-se por sua **relevância teórica** (o problema é **NP-difícil**) e **prática** (aplicações em escalonamento, alocação de registros em compiladores, otimização de redes, entre outras).

2 Fundamentação Teórica: O Problema da Coloração

2.1 Conceitos Básicos

Um grafo $G = (V, E)$ é composto por um conjunto de vértices V e um conjunto de arestas E que conectam pares de vértices. Colorir um grafo significa atribuir rótulos (cores) a seus elementos sob certas restrições.

Definição

[Coloração própria de vértices] Uma função $c : V \rightarrow \mathbb{N}$ é uma coloração própria se, para toda aresta $(u, v) \in E$, tem-se $c(u) \neq c(v)$. Ou seja, vértices vizinhos sempre recebem cores distintas.

Definição

[Número cromático $\chi(G)$] É o **menor número de cores** necessárias para uma coloração própria dos vértices de G . Determinar $\chi(G)$ é um problema **NP-difícil** [2], o que justifica o uso de heurísticas aproximativas em sistemas computacionais como o Grifon.

Definição

[Coloração total] Uma coloração total de G é uma atribuição de cores a $V \cup E$ tal que:

1. Vértices adjacentes têm cores diferentes;
2. Arestas adjacentes (que compartilham um vértice) têm cores diferentes;
3. Todo vértice tem cor diferente das arestas que nele incidem.

O número mínimo de cores necessário para realizar uma coloração total em G é chamado de **número cromático total** ou $\chi_2(G)$ [1]. Determinar esse valor é um problema NP-difícil.

2.2 Teoremas Fundamentais

Dois resultados clássicos orientaram a implementação dos algoritmos de coloração de arestas no Grifon:

Teorema

[Teorema de *Vizing*, 1964] Para qualquer grafo simples G ,

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$$

onde $\Delta(G)$ é o grau máximo de G e $\chi'(G)$ é o índice cromático (número mínimo de cores para arestas).

Isso significa que o algoritmo guloso jamais precisará de mais do que $\Delta + 1$ cores.

Teorema

[Teorema de *König*, 1916] Para grafos bipartidos, o índice cromático é exatamente igual ao grau máximo:

$$\chi'(G) = \Delta(G)$$

O Grifon utiliza esses teoremas tanto para validar seus resultados quanto para escolher estratégias específicas de coloração (como o algoritmo [round-robin](#) para K_5 , que atinge o limite superior de *Vizing*, e o [esquema de diagonais](#) para $K_{3,3}$, que atinge o ótimo de *König*).

3 Algoritmos de Coloração de Vértices

3.1 Algoritmo de *Welsh-Powell* Melhorado

O Grifon adota o **algoritmo de *Welsh-Powell*** como motor principal para coloração de vértices. Trata-se de um método guloso que segue dois passos fundamentais [3]:

1. **Ordenação:** Os vértices são organizados em ordem **decrescente de grau** (número de vizinhos). Nos logs do sistema, isso aparece como:

```

1      Welsh-Powell: ordenando 5 v r tices por grau: A(g=4) ,
      B(g=4) , C(g=4) , D(g=4) , E(g=4)

```

2. **Atribuição de cores:** Para cada vértice na ordem estabelecida, o algoritmo atribui a **menor cor disponível** que não tenha sido usada por nenhum de seus vizinhos já coloridos.

Otimização implementada: O código utiliza a estrutura de dados **Set** (conjunto) para armazenar as cores proibidas, permitindo consultas de pertencimento em $O(1)$. Embora a complexidade geral do algoritmo permaneça $O(V^2)$ – devido à ordenação inicial ($O(V \log V)$) e ao *loop* sobre vizinhos ($O(\text{grau})$) –, essa otimização reduz drasticamente o tempo de execução para grafos densos.

Exemplo prático (K5):

```

1      A (grau 4) → cor 1
2      B (grau 4) → cor 2
3      C (grau 4) → cor 3
4      D (grau 4) → cor 4
5      E (grau 4) → cor 5
6      Welsh-Powell finalizado: 5 cores

```

3.2 Por que *Welsh-Powell* e não *DSATUR*?

O algoritmo *DSATUR* foi proposto por Brélaz (1979) [4] e utiliza o conceito de **grau de saturação** – número de cores diferentes nos vizinhos – para reordenar dinamicamente a prioridade dos vértices a cada iteração. Ao contrário de outros algoritmos construtivos, que inicialmente ordenam os vértices do grafo de acordo com seus graus e os colore iterativamente seguindo a ordenação inicial (Welsh & Powell, 1967), no *DSATUR* a ordem na qual os vértices são coloridos é construída durante o processamento [1]. A ideia consiste em colorir, a cada iteração, o vértice de maior grau de saturação.

Embora o *DSATUR* costume produzir resultados mais próximos do número cromático ideal em grafos complexos, o *Welsh-Powell* foi escolhido para o Grifon por três razões principais:

1. **Previsibilidade em animações:** A ordenação estática inicial permite que o usuário antecipe a sequência de coloração, facilitando o acompanhamento didático.
2. **Menor custo computacional:** O *DSATUR* exigiria recalcular saturações a cada passo, aumentando a complexidade para $O(V^3)$.
3. **Resultado suficiente:** Para os grafos de até 200 vértices suportados pelo sistema, o *Welsh-Powell* produz colorações próximas do ótimo, como demonstrado nos testes com K_5 (5 cores, ótimo) e C_5 (3 cores, ótimo).

4 Algoritmos de Coloração de Arestas

A coloração de arestas no Grifon é **adaptativa**: o sistema analisa a estrutura do grafo e seleciona automaticamente o algoritmo mais adequado. Essa decisão é transparente para o usuário, que apenas observa o resultado final.

4.1 Algoritmo *Round-Robin* (para grafos completos com n ímpar)

Fundamento teórico: Para um grafo completo K_n com n ímpar, o índice cromático é $\chi'(K_n) = n$ (Teorema de *Vizing*). O algoritmo *round-robin* atinge esse ótimo por meio de uma construção geométrica.

Funcionamento:

1. Os n vértices são dispostos em um círculo.
2. Um vértice permanece fixo no centro enquanto os demais são rotacionados.
3. A cada rodada (cor), emparelham-se os vértices opostos no círculo, formando um conjunto de arestas que não compartilham vértices (um "emparelhamento perfeito").
4. Após n rodadas, todas as arestas foram coloridas exatamente uma vez.

Log de execução para K_5 :

```
1 Grafo completo K5 (ímpar) -> usando round-robin
2 Round-robin para K5 (ímpar): 5 cores (ótimo)
3 Arestas coloridas com 5 cores (ótimo)
```

4.2 Esquema de Diagonais (para $K_{3,3}$)

O grafo bipartido completo $K_{3,3}$ (6 vértices, 9 arestas, todos com grau 3) é um caso especial que merece tratamento específico. Pelo Teorema de *König*, $\chi'(K_{3,3}) = \Delta = 3$. O Grifon detecta automaticamente esse grafo (6 vértices, 9 arestas, todos grau 3) e aplica um esquema explícito de diagonais:

- **Cor 1:** arestas $(A1, B1)$, $(A2, B2)$, $(A3, B3)$
- **Cor 2:** arestas $(A1, B2)$, $(A2, B3)$, $(A3, B1)$
- **Cor 3:** arestas $(A1, B3)$, $(A2, B1)$, $(A3, B2)$

Resultado: 9 arestas coloridas com exatamente 3 cores – solução ótima que o algoritmo guloso genérico não conseguiria garantir.

4.3 Algoritmo Guloso Otimizado (caso geral)

Para os demais grafos (caminhos P_n , ciclos C_n , grafos simples etc.), o Grifon utiliza um algoritmo guloso que segue estas etapas:

1. **Construção do grafo de incidência:** Duas arestas são consideradas adjacentes se compartilham um vértice. O sistema constrói uma matriz de adjacência entre arestas, armazenada em um mapa *incidencia* onde cada aresta aponta para um Set de arestas vizinhas.
2. **Cálculo do grau de incidência:** Para cada aresta, conta-se quantas arestas lhe são adjacentes.
3. **Ordenação:** As arestas são processadas em ordem **decrescente de grau de incidência** (priorizando as mais restritivas).

4. **Atribuição de cores:** Para cada aresta, verifica-se as cores já usadas por suas vizinhas e atribui-se a menor cor disponível.

Complexidade: $O(E^2)$, onde E é o número de arestas. Para um grafo completo de 200 vértices ($E \approx 20.000$), isso resulta em cerca de **400 milhões de operações** – ainda viável em navegadores modernos. Para 500 vértices, o número de operações ultrapassa **7,8 bilhões**, causando lentidão perceptível.

Garantia teórica: O algoritmo respeita o Teorema de *Vizing*, utilizando no máximo $2\Delta - 1$ cores, embora na prática frequentemente atinja o limite inferior Δ ou $\Delta + 1$.

5 Arquitetura e Tecnologias do Grifon

O Grifon foi projetado para operar inteiramente no **lado do cliente** (navegador), eliminando a necessidade de servidores ou dependências externas além das bibliotecas padrão.

5.1 Principais Tecnologias

Tecnologia	Função
<i>HTML5 + CSS3</i>	Estrutura e estilo da interface
<i>Bootstrap 5.3</i>	Componentes responsivos (<i>offcanvas</i> , botões, formulários)
<i>Bootstrap Icons</i>	Ícones para ações (adição, remoção, coloração, etc.)
<i>Cytoscape.js</i>	Renderização e manipulação do grafo (<i>zoom</i> , arrasto, seleção)
<i>JavaScript (ES6+)</i>	Lógica de negócio: algoritmos, animações, exportação/importação

Tabela 1: Tecnologias utilizadas no desenvolvimento do Grifon

5.2 Estrutura do Código

O código-fonte está organizado em uma **IIFE** (*Immediately Invoked Function Expression*) para isolar o escopo e evitar conflitos com outras bibliotecas. As principais seções são:

1. **Inicialização do *Cytoscape*:** Configuração do *container*, estilos e comportamento padrão.
2. **Estado global:** Objeto estado que armazena parâmetros como *orientado*, *ponderado*, *velocidade*, *executando*.
3. **Operações básicas:** Funções para adicionar/remover vértices e arestas, renomear, editar pesos.
4. **Algoritmos de coloração:** *Welsh-Powell*, *round-robin*, guloso, detecção de casos especiais.
5. **Animações:** Funções assíncronas que controlam o passo a passo da coloração, com suporte a interrupção.

6. **Eventos do *Cytoscape*:** Cliques, duplo-cliques, clique direito (menu de contexto).
7. **Interface (*UI bindings*):** *Listeners* para botões, *switches* e controles de velocidade.

5.3 Histórico (*Undo/Redo*)

O Grifon implementa um sistema de desfazer/refazer baseado no **padrão de projeto *Command***. Duas pilhas armazenam até 50 ações consecutivas, permitindo ao usuário reverter operações como:

- Adição/remoção de vértices
- Adição/remoção de arestas
- Renomeação de vértices
- Edição de pesos

Os atalhos de teclado **Ctrl+Z** (desfazer) e **Ctrl+Y** (refazer) estão disponíveis, com *feedback* visual no console de *logs*.

6 Funcionalidades da Interface e Usabilidade

6.1 Visão Geral da Interface

A interface do Grifon foi projetada para ser **intuitiva** e **responsiva**, adaptando-se a diferentes tamanhos de tela. Seus principais elementos são:

Elemento	Descrição
<i>Sidebar (offcanvas)</i>	Painel lateral acessível pelo botão flutuante. Contém todas as configurações e ações: tipo de grafo, inserção de vértices/arestas, modelos pré-definidos, exportação/importação.
Barra de ferramentas	Botões flutuantes sobre o grafo para alternar entre modos (inserir vértice, inserir aresta, selecionar).
Painel de <i>logs</i>	<i>Console</i> flutuante, redimensionável e minimizável, que exibe <i>timestamps</i> e mensagens detalhadas da execução dos algoritmos.
Controle de velocidade	Seletor na <i>sidebar</i> que ajusta a animação entre 100ms e 2000ms por passo.
Menu de contexto	Clique direito sobre vértices ou arestas para ações rápidas: renomear, excluir, editar peso, iniciar aresta.

Tabela 2: Elementos da interface do Grifon

6.2 Estatísticas em Tempo Real

Um card informativo na *sidebar* exibe, em tempo real:

- Número de vértices e arestas
- Densidade do grafo ($\frac{2|E|}{|V|(|V|-1)}$ para não orientados, $\frac{|E|}{|V|(|V|-1)}$ para orientados)
- Grau médio ($2|E|/|V|$)
- Número de componentes conexas (calculado via BFS)

6.3 Modos de Operação

Modo	Comportamento	Atalho
Selecionar	Permite clicar e arrastar para mover vértices, selecionar elementos	Padrão
Inserir vértice	Clique no <i>canvas</i> cria novo vértice com ID automático (V1, V2, ...)	Botão na <i>toolbar</i>
Inserir aresta	Clique em um vértice (origem) e outro (destino) cria aresta	Botão na <i>toolbar</i>

Tabela 3: Modos de operação do Grifon

7 Animações e Recursos Didáticos

Um dos pilares do projeto é a **didática visual**. Para isso, o Grifon oferece:

7.1 Animação Passo a Passo da Coloração Total

O usuário pode clicar em "**Colorir (Passo a Passo)**" e acompanhar em tempo real:

- A ordenação dos vértices por grau
- A atribuição de cores a cada vértice
- A construção do grafo de incidência entre arestas
- A atribuição de cores a cada aresta

Velocidade ajustável: O controle deslizante permite variar o atraso entre 100ms (rápido) e 2000ms (muito lento), adequado para demonstrações em sala de aula.

7.2 Centralização Automática

Quando a opção "**Centralizar no vértice atual**" está ativada, o sistema aplica *zoom* e centraliza a visualização no vértice que está sendo colorido no momento, garantindo que o aluno não perca o foco.

7.3 Logs Didáticos

Cada passo da animação é registrado no console de *logs* com:

- *Timestamp* preciso
- Identificação do elemento (vértice ou aresta)
- Cor atribuída
- Contexto (ex.: "proibidas: 1,3"– cores que não puderam ser usadas)

Exemplo de *log* para o *Welsh-Powell*:

```
1 Welsh-Powell: ordem A(g=2), B(g=2), C(g=2), D(g=1), E(g=1)
2 A (grau 2) → cor 1 (proibidas: nenhuma)
3 B (grau 2) → cor 2 (proibidas: 1)
4 C (grau 2) → cor 1 (proibidas: 2)
5 D (grau 1) → cor 2 (proibidas: 1)
6 E (grau 1) → cor 2 (proibidas: 1)
7 Welsh-Powell concluído: 2 cores
```

7.4 Modelos Pré-definidos

O Grifon inclui uma galeria de grafos clássicos para estudo rápido:

Modelo	Descrição	Vértices	Arestas	χ	χ'
K5	Completo com 5 vértices	5	10	5	5
P5	Caminho com 5 vértices	5	4	2	2
C5	Ciclo com 5 vértices	5	5	3	3
K3,3	Bipartido completo	6	9	2	3
Margaridas	Malha de transporte	20	38	2	10
Simples	Quadrado com diagonal	4	5	2	3

Tabela 4: Modelos pré-definidos do Grifon

8 Gestão de Dados e Formato de Arquivo

8.1 Exportação para *.txt*

O Grifon permite salvar o grafo atual em um arquivo de texto com o seguinte formato (exemplificado para K_5):

```
1 5 10
2 0 0
3 A B
4 A C
5 A D
6 A E
7 B C
8 B D
```

```

9 | B E
10 | C D
11 | C E
12 | D E
13 | ---
14 | A 330 250
15 | B 470 350
16 | C 400 450
17 | D 250 400
18 | E 200 300

```

Especificação:

- **Linha 1:** Número de vértices e arestas
- **Linha 2:** Orientação (1 para orientado, 0 para não orientado) e Peso (1 para ponderado, 0 para sem peso)
- **Linhas seguintes:** Lista de arestas (origem destino [peso])
- **Linha '—':** Separador
- **Linhas finais:** Coordenadas (x, y) de cada vértice

8.2 Importação de *.txt*

O processo de importação é simétrico à exportação. O sistema:

1. Lê as configurações de orientação e peso
2. Recria vértices e arestas
3. Restaura as posições originais dos vértices
4. Aplica *zoom* para ajustar a visualização

8.3 Pasta */data*

O diretório */data* contém arquivos de exemplo no formato especificado, permitindo que o usuário carregue rapidamente grafos pré-definidos ou compartilhe seus próprios modelos.

9 Performance e Limitações Técnicas

O Grifon foi otimizado para uso educacional e **grafos de até 200 vértices**. Acima desse limite, o desempenho pode degradar-se significativamente devido à complexidade dos algoritmos e ao consumo de memória.

9.1 Complexidade dos Algoritmos

Algoritmo	Complexidade	Comportamento em K_{200}
<i>Welsh-Powell</i> (vértices)	$O(V^2)$	$\approx 40\text{ms}$
<i>Round-robin</i> (arestas, K_n ímpar)	$O(V^2)$	$\approx 5\text{ms}$
Guloso (arestas, caso geral)	$O(E^2)$	$\approx 800\text{ms}$
Animção passo a passo	$O(V + E)$ por passo	Fluída

Tabela 5: Complexidade dos algoritmos implementados

9.2 Consumo de Memória

Nº vértices	Arestas (pior caso)	Memória estimada
200	19.900	≈ 50 MB
500	124.750	≈ 200 MB
1.000	499.500	≈ 800 MB
2.000	1.999.000	≈ 3 GB
5.000	12.497.500	≈ 20 GB (navegador <i>crash!</i>)

Tabela 6: Consumo de memória por tamanho do grafo

9.3 Gargalo do Algoritmo Guloso

O principal gargalo é o *loop* aninhado que constrói o grafo de incidência entre arestas:

Listing 1: Loop aninhado do algoritmo guloso

```
1 for (let i = 0; i < arestas.length; i++) {  
2   for (let j = i + 1; j < arestas.length; j++) {  
3     // verifica se as arestas compartilham um v rtice  
4   }  
5 }
```

Para um grafo completo com 200 vértices ($E \approx 20.000$), esse *loop* executa cerca de **200 milhões de iterações**. Para 500 vértices, o número sobe para **7,8 bilhões**, tornando a operação inviável em navegadores comuns.

Por essa razão, o sistema limita a criação de vértices a **200** e recomenda o uso de grafos esparsos para demonstrações interativas.

10 Aplicações Práticas da Coloração de Grafos

A coloração de grafos, embora seja um problema clássico da matemática discreta, possui inúmeras aplicações no mundo real [1]. O Grifon serve como plataforma para explorar essas aplicações de forma visual e interativa.

10.1 Alocação de Registros em Compiladores

Em compiladores, as variáveis de um programa são representadas como vértices em um **grafo de interferência**: duas variáveis são conectadas por uma aresta se seus ciclos de vida se sobrepõem (não podem ser armazenadas no mesmo registrador). O número cromático desse grafo indica o número mínimo de registradores necessários [5]. O Grifon pode ser usado para visualizar e colorir grafos de interferência de pequenos trechos de código.

10.2 Escalonamento de Horários (Problema do Horário Escolar)

O problema de alocar horários para disciplinas sem conflitos (um professor não pode estar em duas salas ao mesmo tempo, um aluno não pode fazer duas provas simultâneas) é modelado como coloração de vértices. O Grifon permite que o usuário construa o grafo de conflitos e encontre uma coloração viável.

10.3 Malha de Transporte Público e Redes de Internet

Duas aplicações importantes e visualmente didáticas da coloração de grafos são a **otimização de rotas de transporte público** e o **roteamento de pacotes em redes de internet**.

No transporte público, estações (vértices) e linhas (arestas) formam um grafo que pode ser colorido para identificar conflitos de horários, sobreposição de rotas e alocação de veículos. O modelo “**Grafo Margaridas**”, incluso no Grifon, simula uma malha de transporte com dois terminais centrais conectados a diversas linhas periféricas, permitindo visualizar como a coloração total pode auxiliar no planejamento urbano.

Em redes de computadores, roteadores e *switches* são vértices, e as conexões entre eles são arestas. A coloração de arestas ajuda a evitar colisões de dados e a alocar canais de comunicação sem interferência. O problema de atribuir **canais Wi-Fi** para roteadores próximos é um exemplo clássico de coloração de grafos, onde roteadores vizinhos não podem operar na mesma frequência.

Ambas as aplicações podem ser simuladas e visualizadas passo a passo no Grifon, tornando o aprendizado mais concreto e significativo.

10.4 Atribuição de Frequências em Redes de Celular

Torres de celular próximas não podem operar na mesma frequência para evitar interferência. O problema de atribuir frequências minimizando o número total é uma coloração de grafos [6]. O Grifon pode simular pequenas redes e demonstrar o uso de algoritmos gulosos para esse fim.

10.5 Otimização de Tráfego Aéreo

Em aeroportos, pousos e decolagens devem ser escalonados para evitar colisões. O problema é modelado como coloração de arestas, onde cada aresta representa um voo e arestas concorrentes (que usam a mesma pista em horários próximos) não podem ter a mesma cor [7].

11 Conclusão

O **Grifon - Grafo Interativo v3.6.1** cumpre integralmente os objetivos propostos no Seminário II da disciplina de Algoritmos em Grafos. O sistema oferece:

1. **Implementação robusta** dos principais algoritmos de coloração: *Welsh-Powell* para vértices, *round-robin* para K_n com n ímpar, esquema específico para $K_{3,3}$ e algoritmo guloso otimizado para os demais casos.
2. **Interface amigável e responsiva**, com painel de *logs* flutuante, controle de velocidade, modos de operação e menu de contexto.
3. **Animações didáticas** que permitem acompanhar passo a passo a execução dos algoritmos, com destaque visual para o elemento sendo colorido.
4. **Portabilidade e interoperabilidade** através da exportação/importação de grafos em formato `.txt`.
5. **Modelos pré-definidos** de grafos clássicos (K_5 , P_5 , C_5 , $K_{3,3}$, Margaridas, Simples), permitindo testes rápidos e demonstrações.
6. **Documentação** (código escrito e comentado em português, *logs* descritivos, *RE-ADME*) que facilita a manutenção e futuras extensões.

Trabalhos futuros:

- Implementação do algoritmo de correspondência para colorir otimamente **qualquer** grafo bipartido (não apenas $K_{3,3}$).
- Suporte a grafos planares com o algoritmo de 4 cores.
- Exportação em formatos padrão (*GraphML*, *LaTeX/TikZ*) para uso em publicações acadêmicas.
- Modo "comparação" que exibe lado a lado os resultados de diferentes algoritmos (ex.: *Welsh-Powell* vs. *DSATUR*).

Em síntese, o Grifon não apenas atende a todos os requisitos do seminário, mas também se consolida como uma ferramenta útil e didática para o ensino de algoritmos em grafos, servindo de base para futuros trabalhos na área.

Desenvolvido por: *Jailis Dourado*
Disciplina: Algoritmos em Grafos (7º período)
Instituição: Instituto Federal Goiano
Versão: 3.6.1
Ano: 2026

 Acesse o Projeto

github.com/guardiaojailis/grifon-color

Referências

- [1] GOLDBARG, Marco; GOLDBARG, Elizabeth. **Grafos: conceitos, algoritmos e aplicações**. Rio de Janeiro: Elsevier, 2012. 474p. ISBN 978-85-352-5716-8.
- [2] KARP, Richard M. Reducibility among combinatorial problems. In: **Complexity of Computer Computations**. New York: Plenum Press, 1972. p. 85-103.
- [3] WELSH, D. J. A.; POWELL, M. B. An upper bound for the chromatic number of a graph and its application to timetabling problems. **The Computer Journal**, v. 10, n. 1, p. 85-86, 1967.
- [4] BRÉLAZ, Daniel. New methods to color the vertices of a graph. **Communications of the ACM**, v. 22, n. 4, p. 251-256, 1979.
- [5] CHAITIN, Gregory J. Register allocation and spilling via graph coloring. **ACM SIGPLAN Notices**, v. 17, n. 6, p. 98-105, 1982.
- [6] AARDAL, K. I. et al. Models and solution techniques for frequency assignment problems. **ZIB-Report 01-40**. Berlin: Konrad-Zuse Zentrum für Informationstechnik, 2001.
- [7] BARNIER, Nicolas; BRISSET, Pascal. Graph coloring for air traffic flow management. **Annals of Operation Research**, v. 130, p. 163-178, 2004.
- [8] VIZING, V. G. On an estimate of the chromatic class of a p-graph. **Discrete Analysis**, v. 3, p. 25-30, 1964.
- [9] KÖNIG, D. Über Graphen und ihre Anwendung auf Determinantentheorie und Mengenlehre. **Mathematische Annalen**, v. 77, p. 453-465, 1916.
- [10] ZYKOV, A. A. On some properties of linear complexes. **Matematicheskii Sbornik**, v. 24, n. 2, p. 163-188, 1949.
- [11] APPEL, K.; HAKEN, W. Every planar map is four-colorable. **Illinois Journal of Mathematics**, v. 21, p. 429-567, 1977.
- [12] CHUDNOVSKY, M. et al. The strong perfect graph theorem. **Annals of Mathematics**, v. 164, p. 51-229, 2006.